Paper Review • Embodied AI / Embodied RSI

PyRUA-REAL

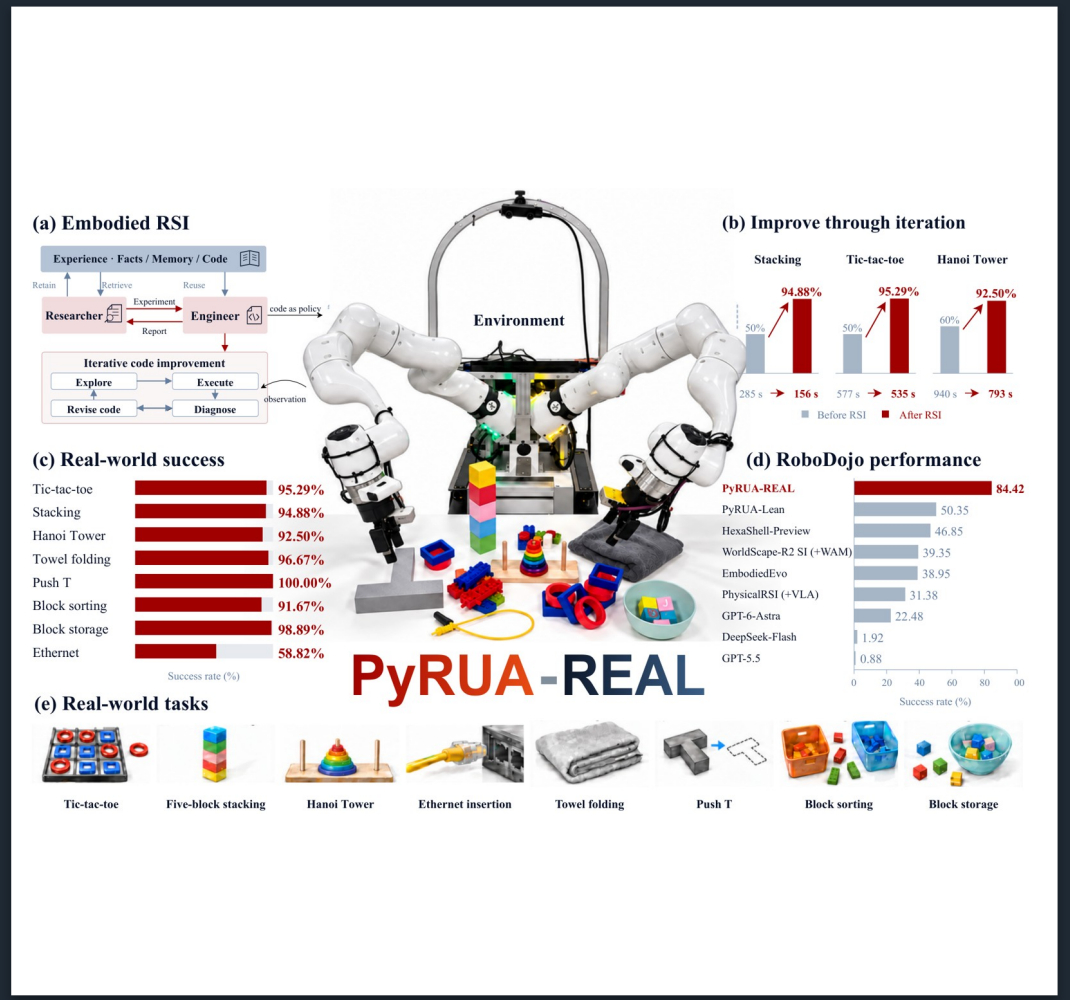
Robot Exploration and Adaptive Learning through RSI in the Real World

Agents try and recode on real robots, and freeze lessons into LLM-free code

Qiang Wang*, Jianxin Bi* ‡ , Shulong Jiang*, Ruiyang Si, Wenbo Huang, Kaiqi Chen, Jiaxin Li, Liyun Li, Xuanyi Li † , Daquan Zhou †

Peking University • EngineAI • National University of Singapore

arXiv 2611.06768 | First submitted 2026-11-04



Source: paper, Figure 1

ONE-LINER

In one line: real-robot trials → accumulated experience → LLM-free programs

PyRUA-REAL lets two coding agents, a Researcher and an Engineer, run experiments, diagnose failures and edit code directly on a real dual-arm Franka. Experience is stored in three memory tiers (facts / lessons / reusable code), and each task is finally frozen into a program that **calls no LLM at runtime**.

7 / 8
Real-robot tasks yielding agent-free programs

91.1%
8 real-robot tasks
Mean dev. success rate*

84.4%
RoboDojo 42-task success
（PyRUA-Lean 50.35%）

5–11×
Executes faster than online coding / online action inference

* 91.1% averages 7 offline tasks with the 58.82% "online-assisted" cable-insertion rate; success rates pool trials across multiple code versions during development and do not reflect repeated-test reliability of a single frozen program.

RESEARCH QUESTION

Key question: how can physical experience become ever-improving robot skills?

Background

Coding agents such as GPT-6-Astra can turn spatial reasoning into executable code and revise it from execution feedback.

The paper asks: with model parameters fixed, can physical interaction experience become robot capability that persists and improves across attempts?

Formulation

$$a_t = \pi_\tau(o_{\leqslant t}; \mathcal{L})$$

Task program π_τ + shared skill library $\mathcal{L}$

$$(\pi^{k+1}, \mathcal{L}^{k+1}, \mathcal{M}^{k+1}) = U_\theta(\tau, \pi^k, \mathcal{L}^k, \mathcal{M}^k, E^k)$$

θ fixed; E^k: execution evidence; $\mathcal{M}$: experience store; maximize expected performance within interaction and compute budgets

① Hard decisions

Which experiment to run? What to change? When to abandon a route? Repeated local fixes may preserve unsuitable perception or control designs.

② Ambiguous feedback

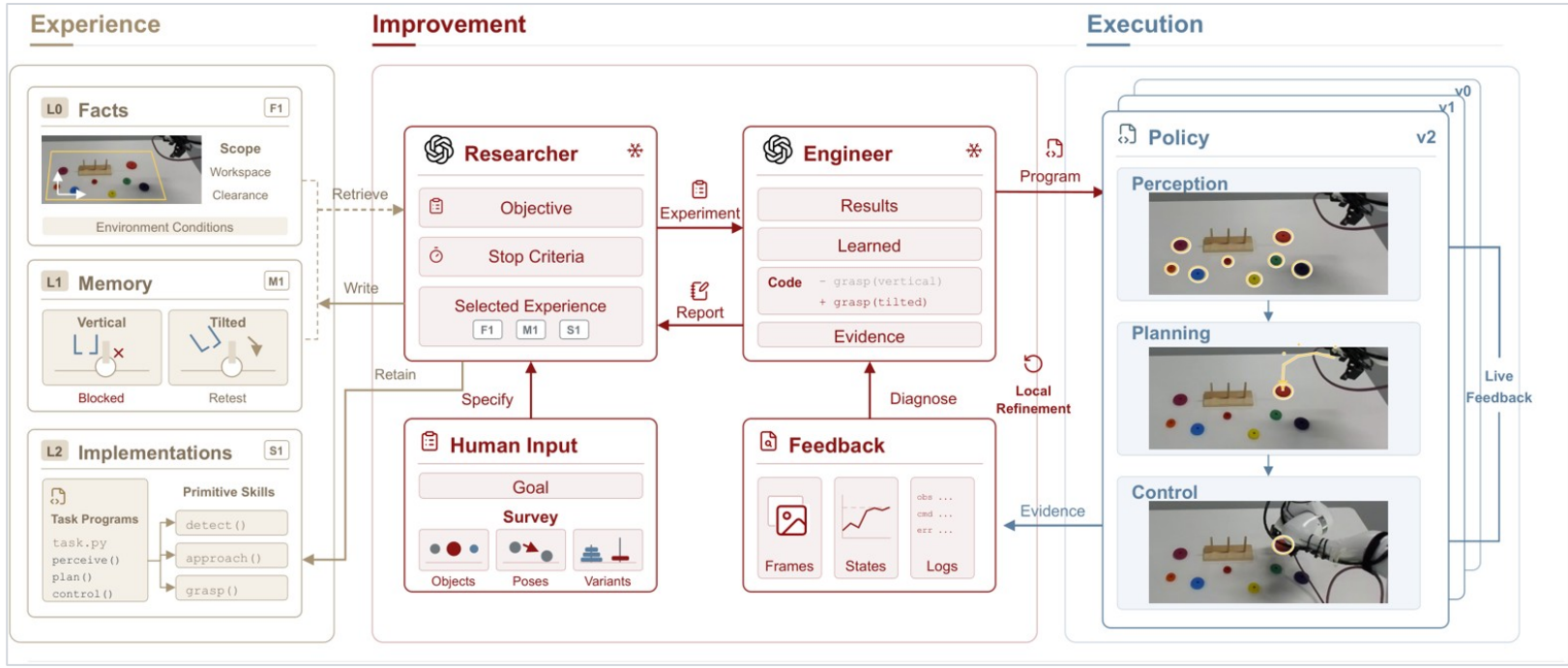
A "failure" may stem from motion, perception, or the verifier that judges success itself.

③ Generalization & reuse

Success in one setup ≠ robustness to position, orientation and task variants; can solutions transfer to new tasks and run without an online agent?

METHOD • SYSTEM

Framework: Researcher plans, Engineer tests, experience accrues, programs run solo



Improvement

Researcher: goal / stop criteria / curated experience

Engineer: results / findings / code / evidence

Experience

L0 Facts • L1 Memory • L2 Implementation

Both successful and failed findings are kept

Execution

Frozen program: perception → planning → control

No LLM at runtime; closed loop on real-time feedback

Humans give only the task goal + a variation survey; agents build calibration, controllers and solvers themselves.

Experiments as the unit of collaboration; experience in three tiers

Researcher

Views the whole task and required variants

Picks the technical route; decides to refine, pivot or advance

→

Experiment spec

Goal

Stop: achieved / hypothesis refuted / bottleneck / no progress

Curated experience: facts, past solutions, skills

→

Engineer

Codes, runs on the robot, tests and diagnoses locally

Can test one step, not the full task each time

→

Report

results: outcome & termination reason

learned: findings

evidence: execution logs

code: final implementation

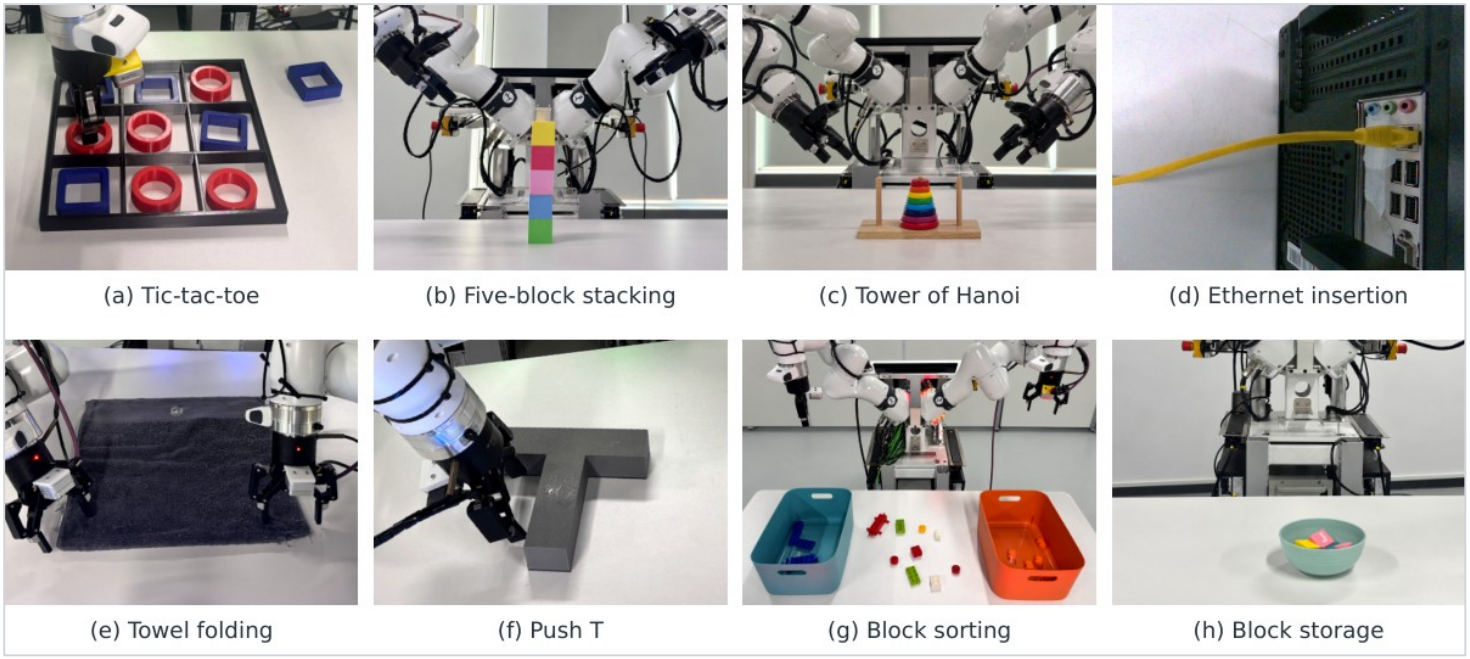
↺ **Researcher reads report → refine current approach / change direction / advance to next stage, then define next experiment**

L0 Facts	L1 Memory	L2 Implementation
Stores: robot, workspace, operating conditions	**Stores:** problems met, approaches tried, outcomes, failure causes	**Stores:** task programs & reusable primitives (detect / approach / grasp…)
Use: Researcher: understand relevant conditions	**Use:** Researcher: retrieve related past tasks, approaches and skills	**Use:** Engineer: inspect, adapt and test implementations

Skill usage counts only each experiment's final implementation; the Researcher decides which adapted implementations become new skills and records their scope and limitations.

SETUP

8 real-robot tasks: reasoning, long-horizon, precision, deformable, bimanual



(a) Tic-tac-toe (b) Stack 5 (c) Hanoi (d) Cable insert (e) Towel fold (f) Push T (g) Sort (h) Tidy

Real-robot platform

2× Franka Research 3 + Robotiq-85 grippers

Head ZED Mini + wrist RealSense D405

Precision	Stacking • Hanoi • Cable • Push T
Long-horizon	Tic-tac-toe • Stacking • Hanoi
Reasoning	Tic-tac-toe • Sorting
Contact-rich	Cable • Push T
Deform./ bimanual	Towel folding
Multi-object	Tic-tac-toe • Stack • Hanoi • Sort • Tidy

Simulation: RoboDojo, 42 tasks

Gen. 12 • Memory 6 • Fine 8 • Long-hor. 8 • Open 8

Agent-free programs for 7/8 tasks; first success took 1.5 h to 45 h

Task	Time to 1st success	Dev. success rate	Median exec. time
Tic-tac-toe	~9 h	**95.29%**	539 s
5-block stack	~2 h	**94.88%**	179 s
Tower of Hanoi	~22 h	**92.50%**	1003 s
Towel folding	~5 h	**96.67%**	30 s
Push T	~45 h	**100%**	129 s
Sorting	~4 h	**91.67%**	179 s
Tidying	~1.5 h	**98.89%**	109 s
Cable insert (offline)	240 h, no success	**0%**	—
Cable insert (online Astra)	+ ~15 h more	**58.82%**	519 s

How is 91.1% computed?

= (7 offline task rates + cable online-assisted 58.82%) ÷ 8

≈ 83.7%

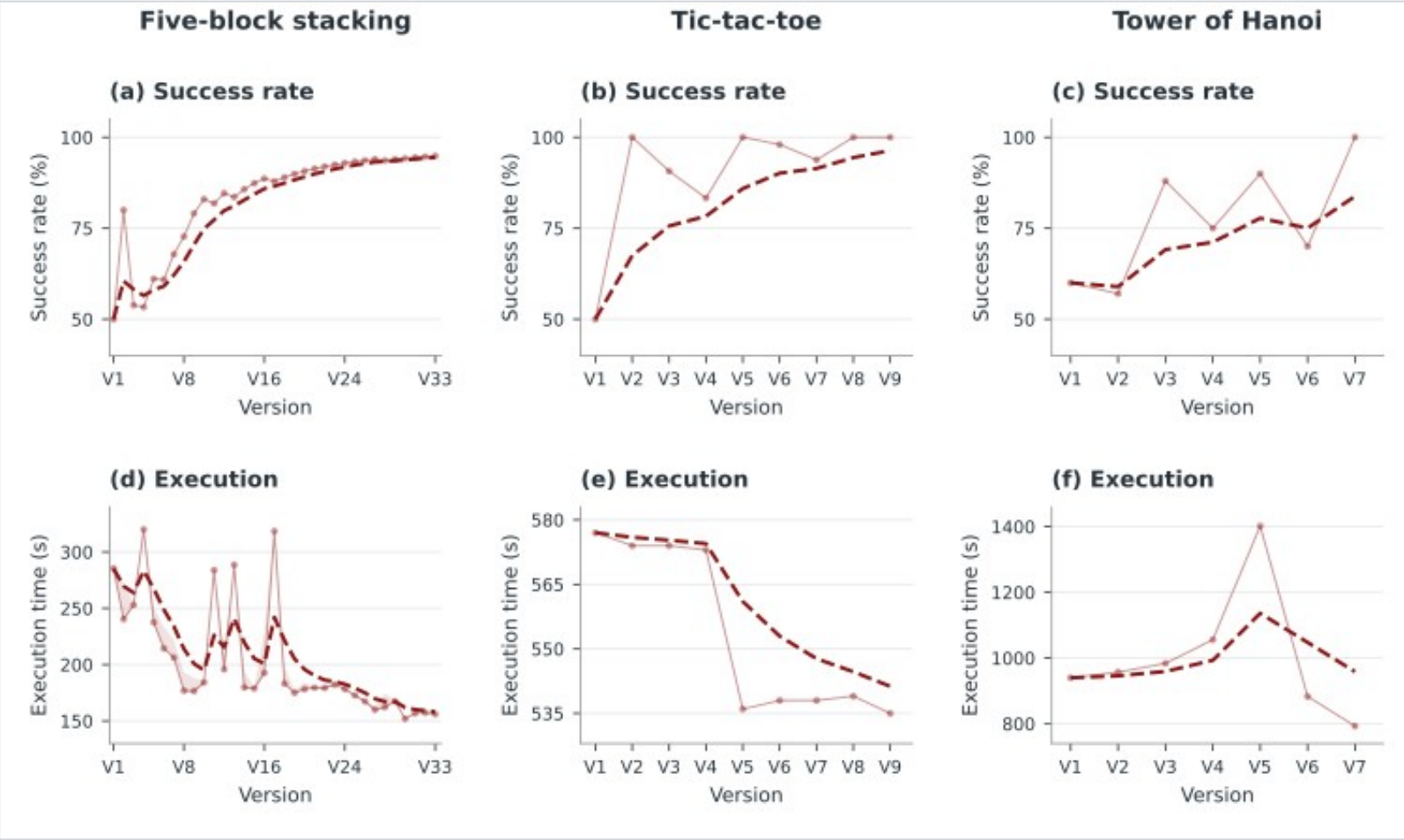
Cable insertion as offline 0%

≈ 95.7%

7 offline tasks only

All rates pool trials across code versions during development; they do not equal one frozen program's reliability.

Still improving after first success: stacking cycle 285 s → 156 s



Stack 5 • incremental tuning

Cycle 285.3 → 156.0 s

Lat. error 2.69→0.44 mm; angle 1.37°→0.25°

Stage success 88.7%→98.7%

Tic-tac-toe • motion strategy switch

V4→V5: multi-segment high waypoints → one JointMotion

573 → 536 s; success 50% → 100% (with dips)

Hanoi • robust first, then fast

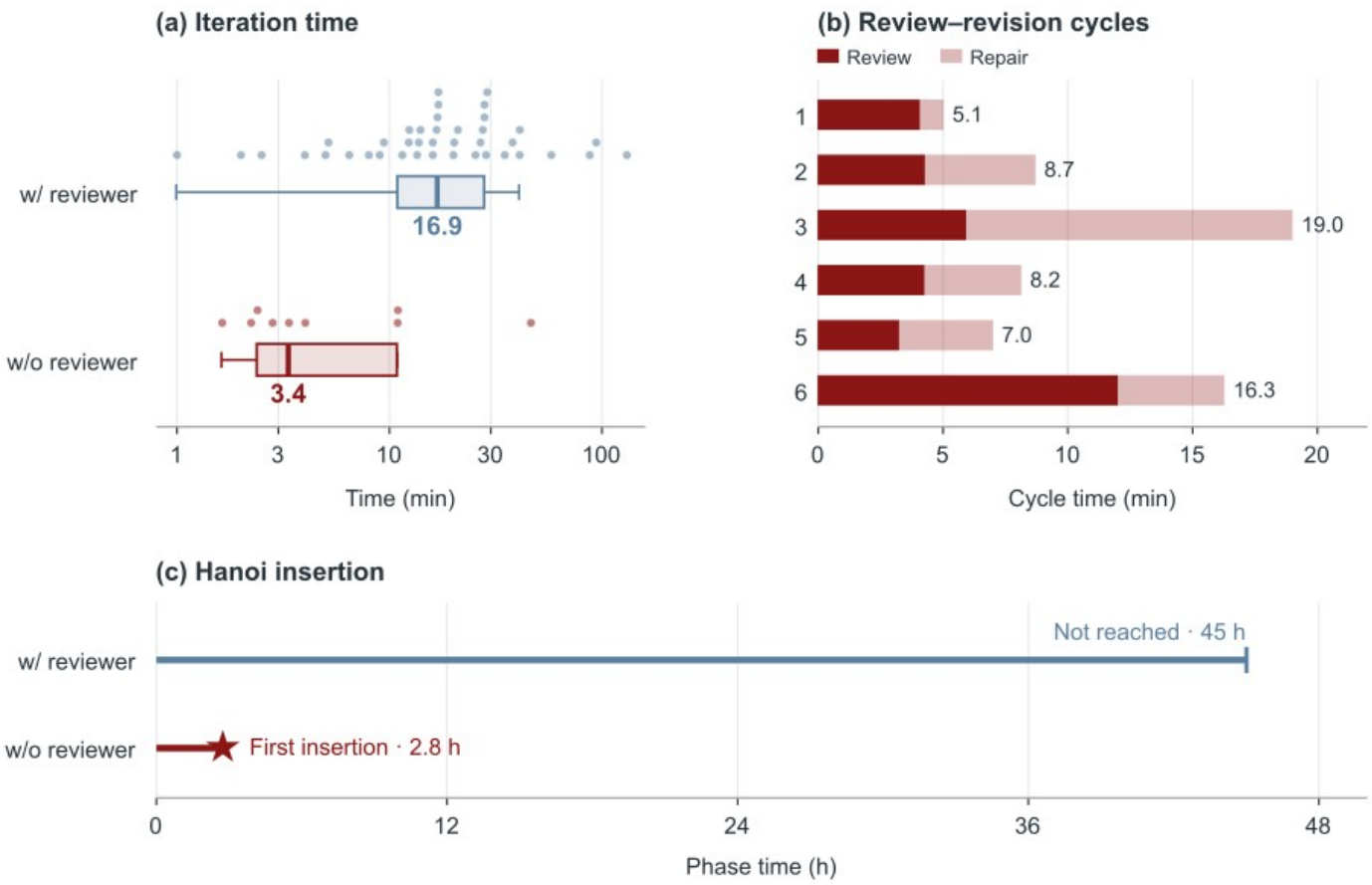
Robust stage 1402 s → faster versions 883 / 793 s

Success 60% → 100%

Development data, not repeated frozen-program tests; errors are visual estimates.

H2 • COORDINATION COST

Adding a veto-only reviewer agent slows exploration



16.9
With reviewer
Median interval (min)

3.4
No reviewer
Median interval (min)

Veto–revise cycles in Hanoi

6 cycles, 5.1–19.0 min each, median 8.5 min

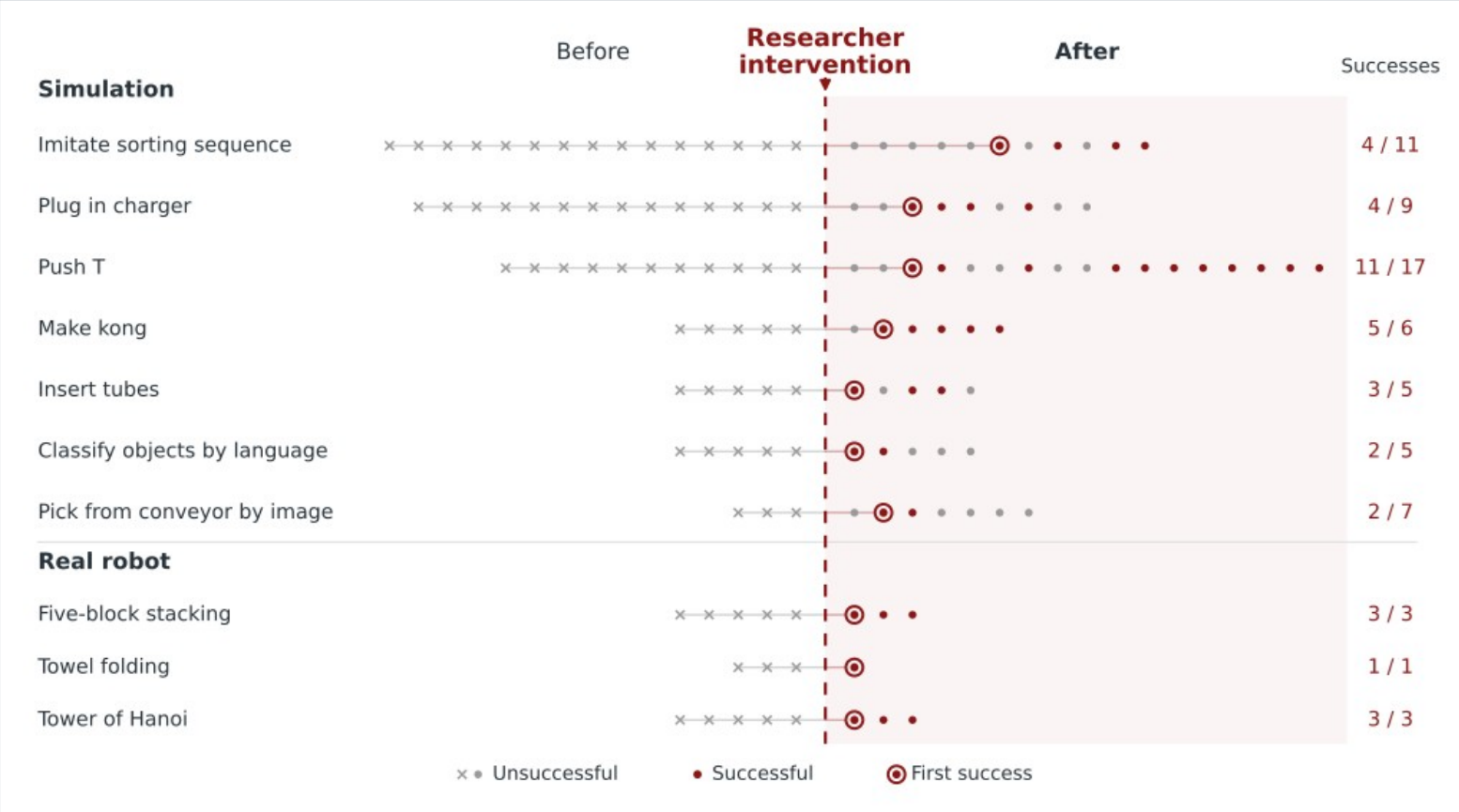
45 h no insertion vs 2.8 h to 1st

Phases ran sequentially; the latter benefited from prior exploration, so gains can't be attributed solely to removing the reviewer.

Robust finding: the approval role slowed acquisition of new physical evidence.

H2 • TASK-LEVEL REASSESSMENT

Researcher route re-evaluation breaks local-fix loops



3/3 • 1/1 • 3/3

Real robot: stacking • towel • Hanoi
Next trial after re-evaluation succeeded

Mixed in simulation

Push T 11/17 • Make kong 5/6

Imitate sorting 4/11 • Pick from conveyor 2/7

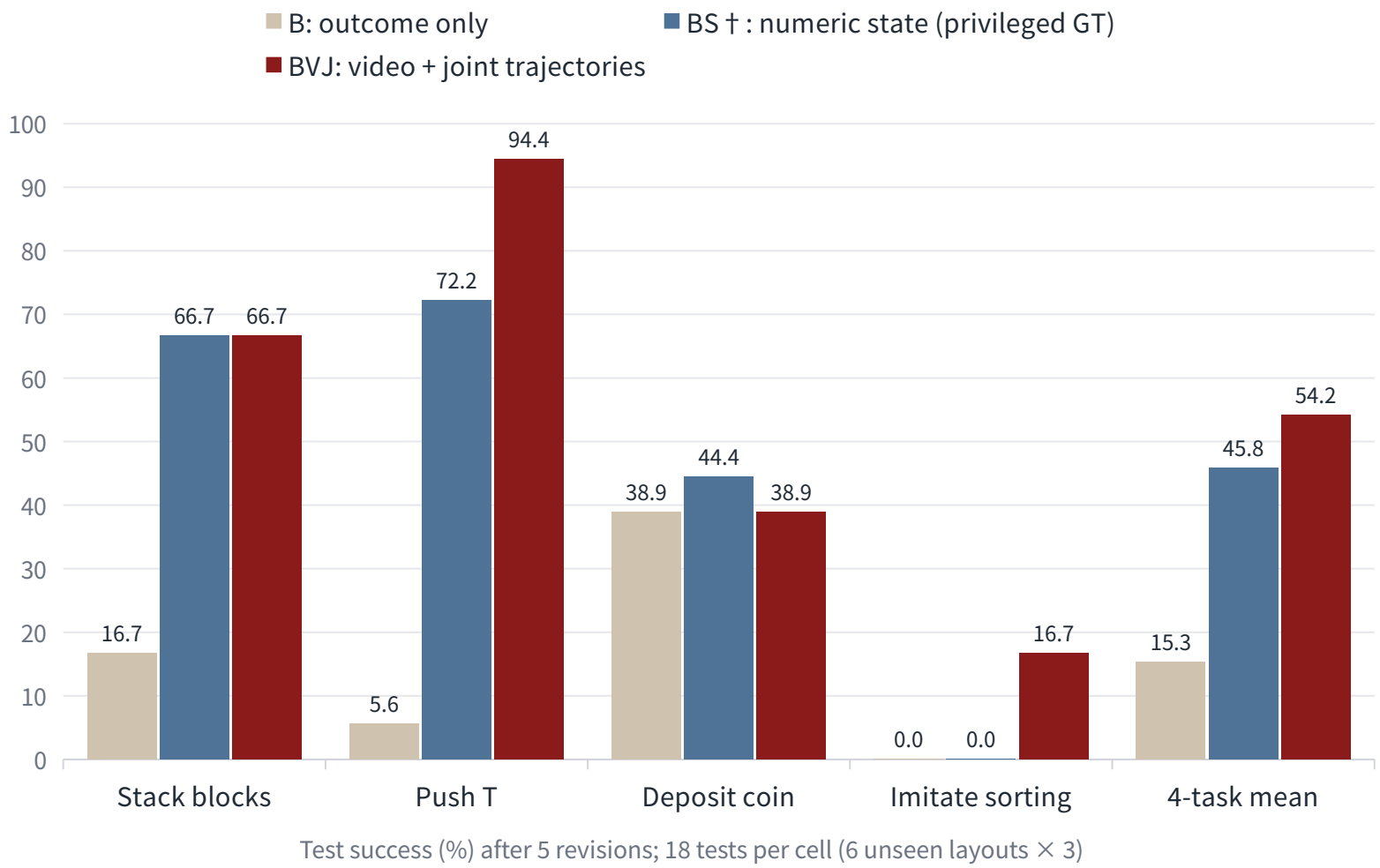
Takeaway

Cap local fixes; hand unresolved failures back to the Researcher to re-plan the route.

(Correlational: not separated from continued exploration)

H3 • EXECUTION FEEDBACK

Richer feedback, better fixes: video + joints 54.2% vs outcome-only 15.3%



Test success (%) after 5 revisions; 18 tests per cell (6 unseen layouts × 3)

54.2% vs 15.3%

BVJ vs B mean test success

Richer, not costlier

Optimizer time 25.4 vs 31.8 min

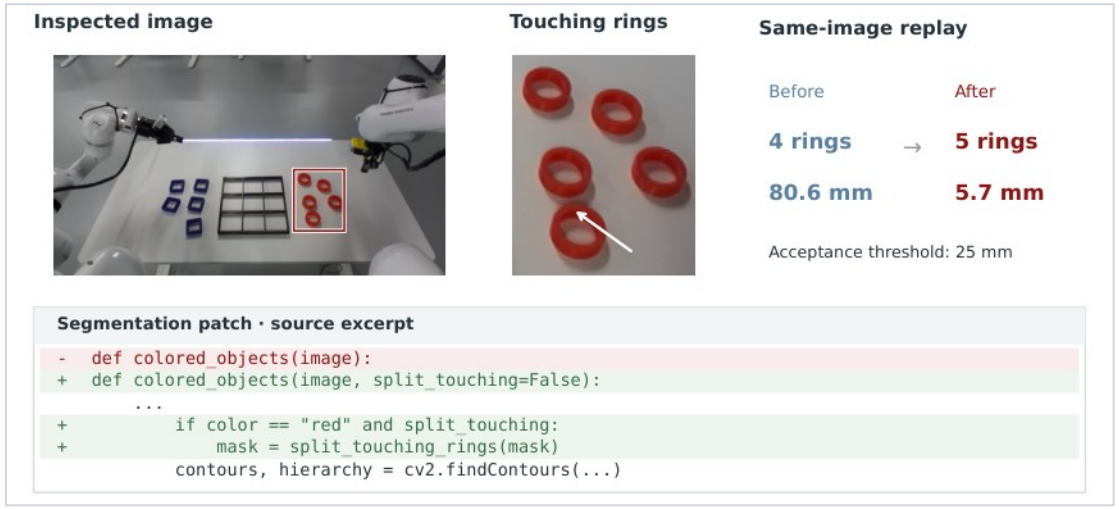
Tokens 3.65M vs 3.54M (similar)

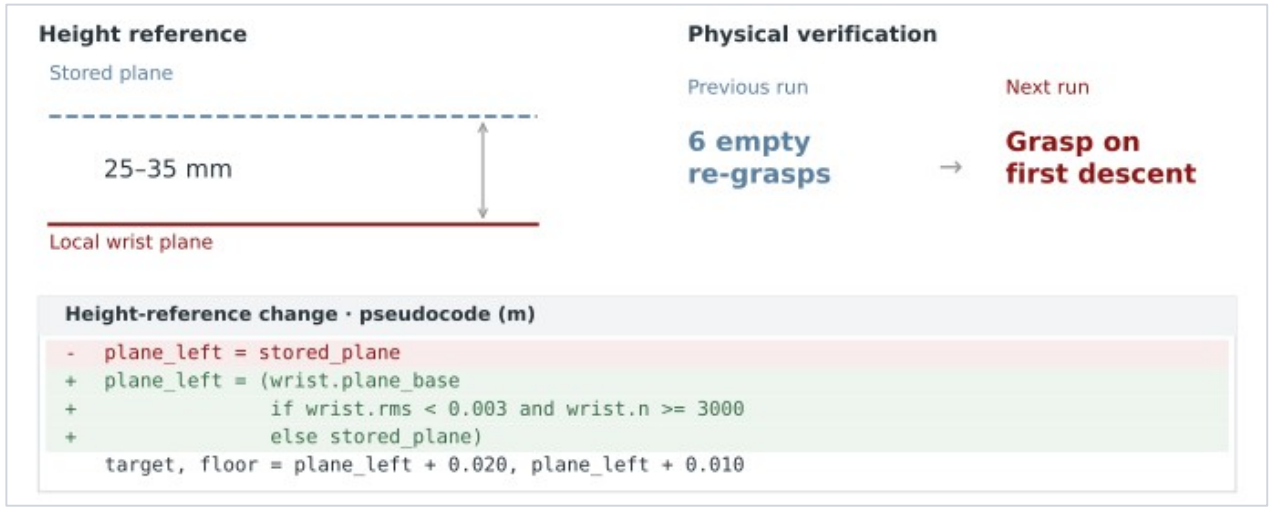
Caveats

Gains vary by task: Push T 5.6→94.4%; Deposit coin flat

1 session per condition; variance unmeasured

Many "failures" are measurement errors: check execution vs. verification first





Tic-tac-toe: detector merges two touching rings

4 rings detected (5 actual) → 80.6 mm error > 25 mm threshold; false failure

Split touching red rings → replay: 5 rings, 5.7 mm

Then 50 regression checks + 178-frame static perception check

Verifier fixed; robot motion unchanged

Towel: table height set too high → empty grasps

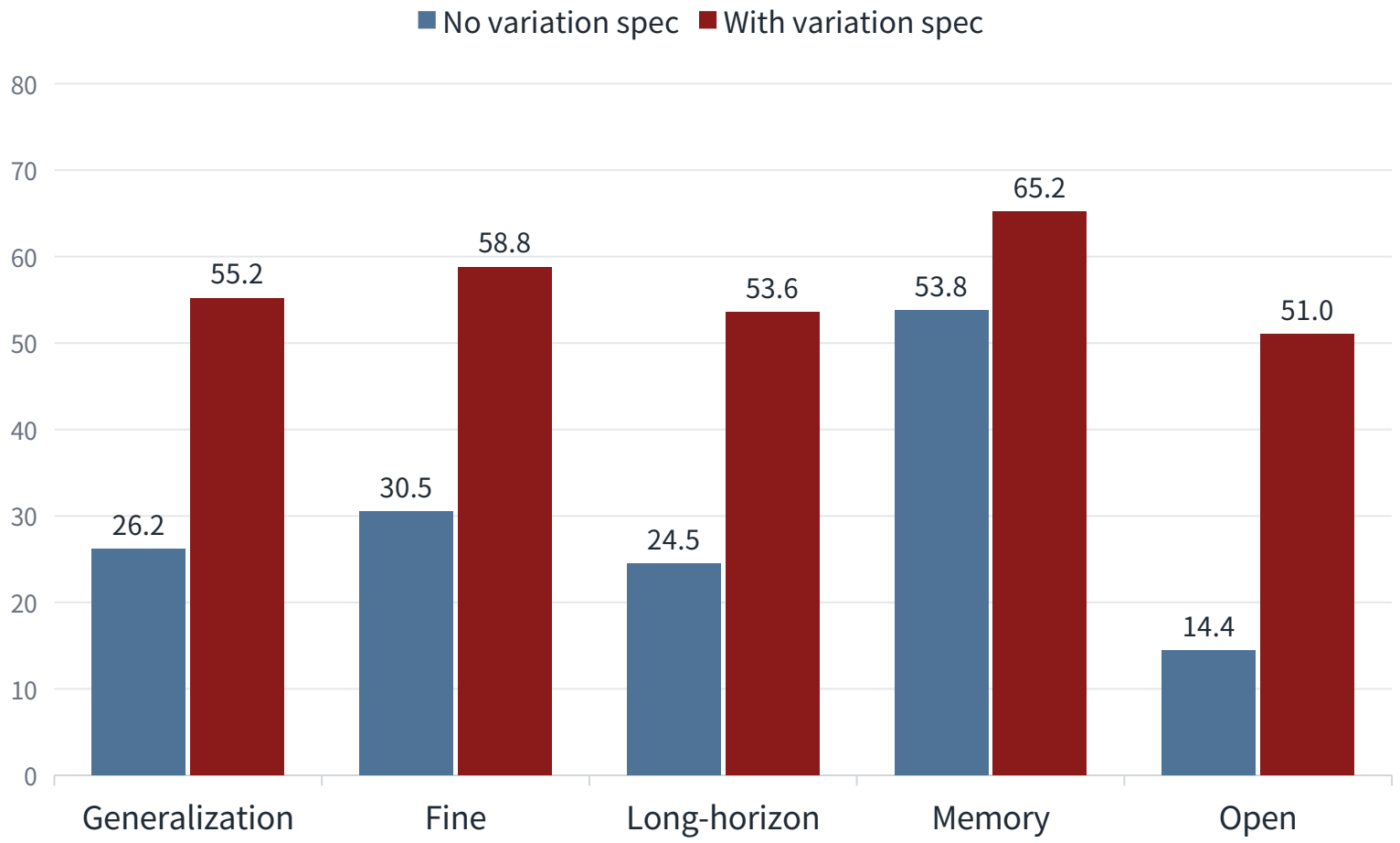
Stored plane 25–35 mm above wrist local fit → 6 regrasps all missed

Use wrist plane when local-fit RMS < 3 mm and ⩾ 3000 points

Next run grasped on first descent (one recovery, not a repeated rate)

H4 • EXPLICIT TASK REQUIREMENTS

Specifying task variation lifts unseen-layout success in all five categories



Unseen-layout success (%); gains (pp): Gen. +28.9 • Fine +28.3 • Long-hor. +29.1 • Memory +11.4 • Open +36.6

+11.4 ~ +36.6 pp

All five categories improve

Per-task example

Stack bowls 0.0 → 86.4%

Fasten screws 9.1 → 81.8%

General pickup 0.0 → 74.4%

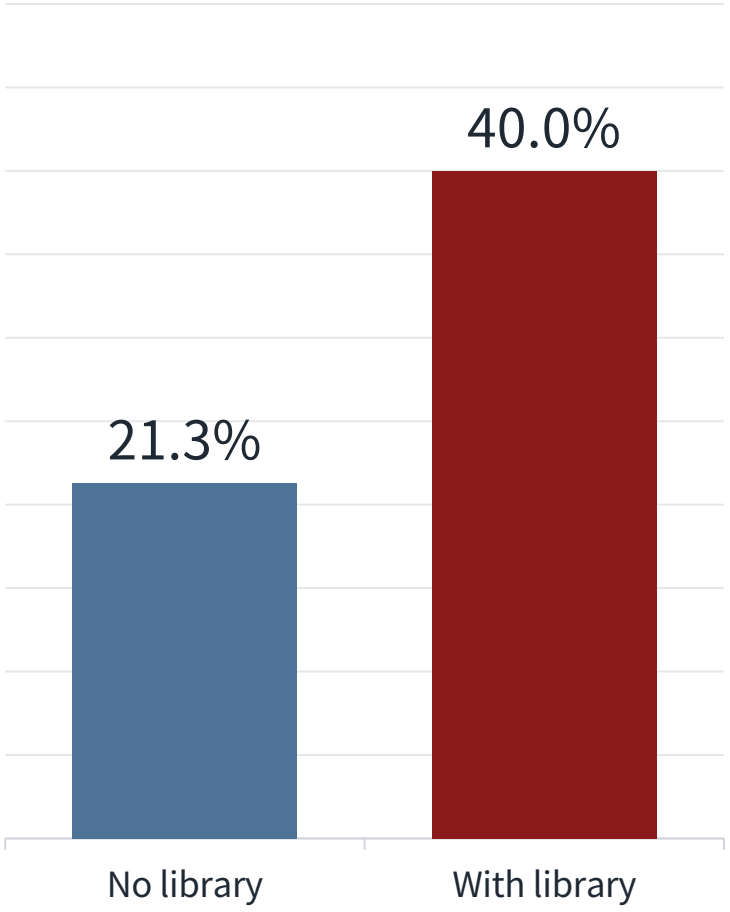
Counterexample: Swap blocks 68.2 → 59.1%

Caveats

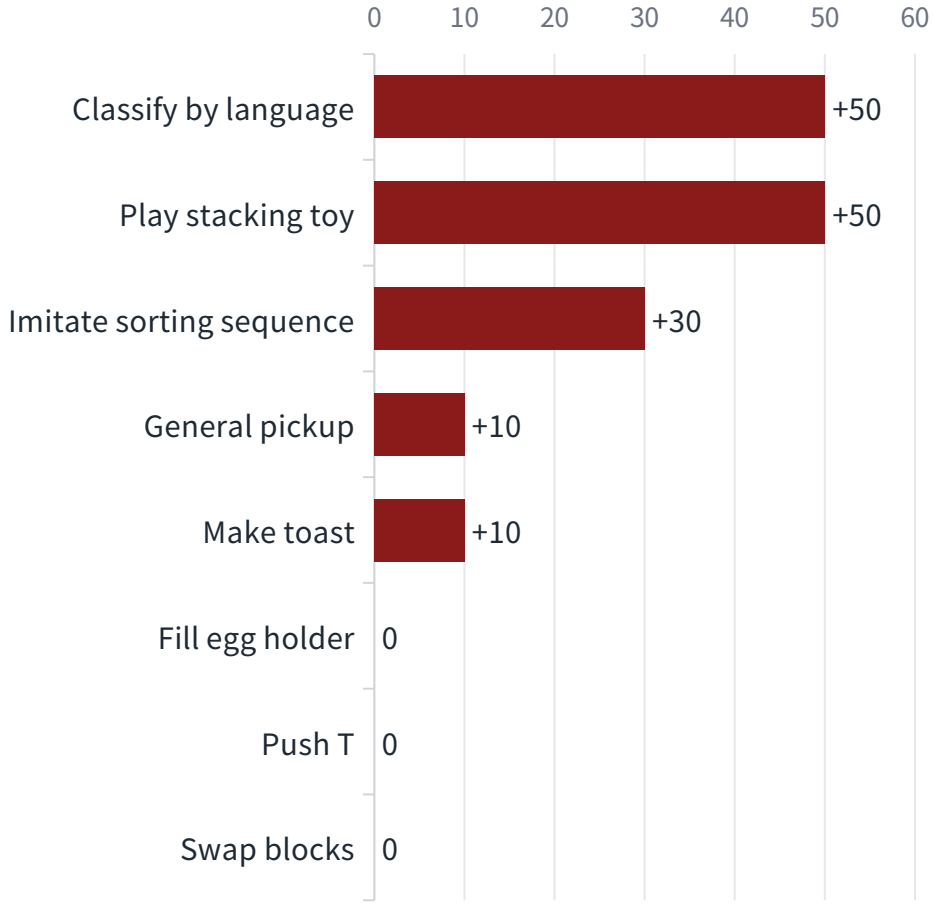
Later version also had extra exploration; spec effect not isolated (32 tasks)

H5 • CROSS-TASK TRANSFER

Reusing prior-task primitive skills: new-task mean success 21.3% → 40.0%



Mean success over 8 target tasks



Per-task gain = with library − without (pp)

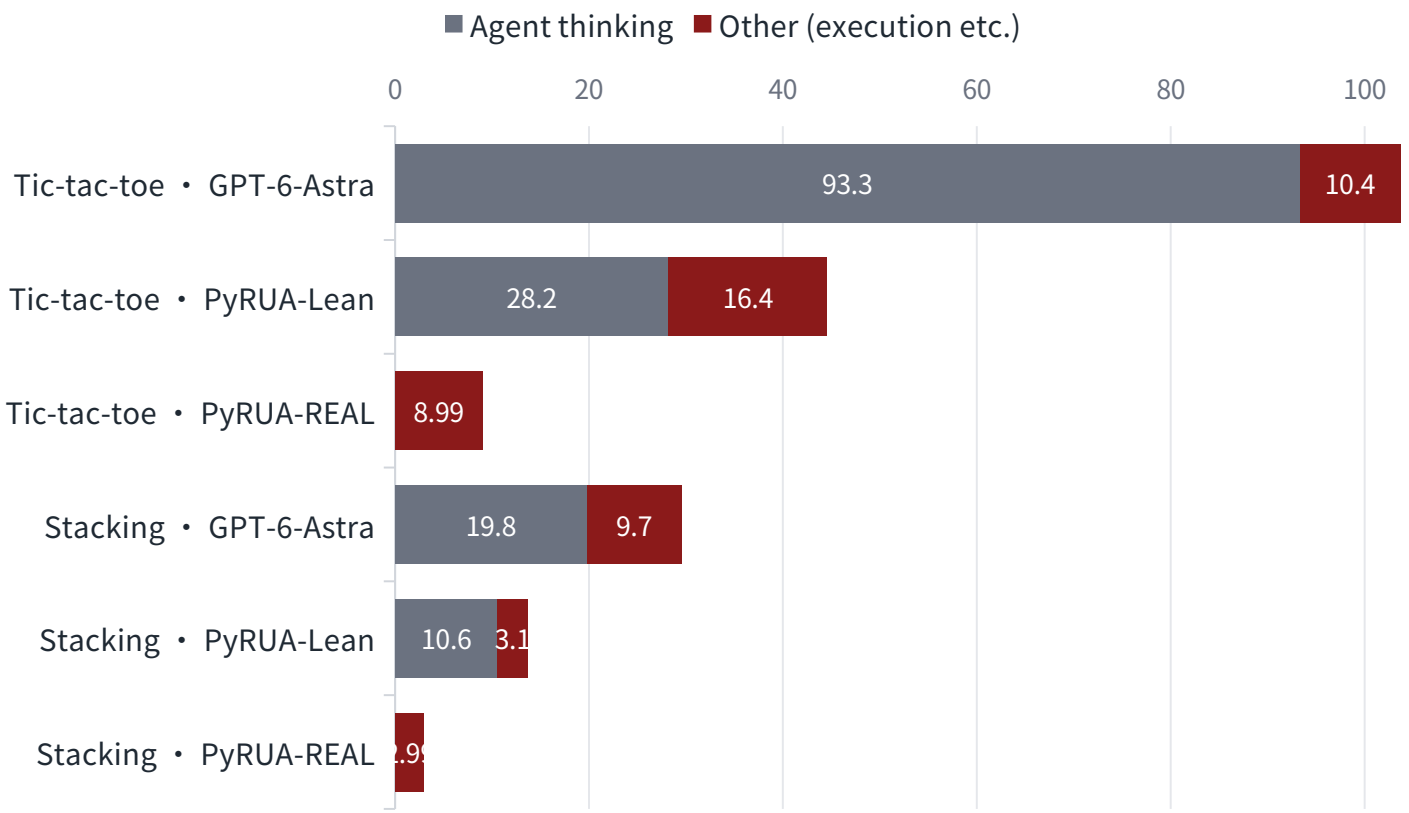
5 / 8
new tasks benefit

0
tasks show negative transfer

Gains vary: language sorting, toy stacking +50 pp; Push T and 2 others unchanged.

H6 • DEPLOYMENT EFFICIENCY

Frozen programs run 5–11× faster; online time is spent mostly thinking



Execution time (min, incl. reset); offline: median of successes, online: logged; excl. development

Hanoi: both online baselines unfinished in 120 min; PyRUA-REAL offline program 16.72 min

4.6–5.0×
PyRUA-Lean
Time ratio

9.9–11.5×
GPT-6-Astra
Time ratio

63–90%
Share of "agent thinking" in online execution

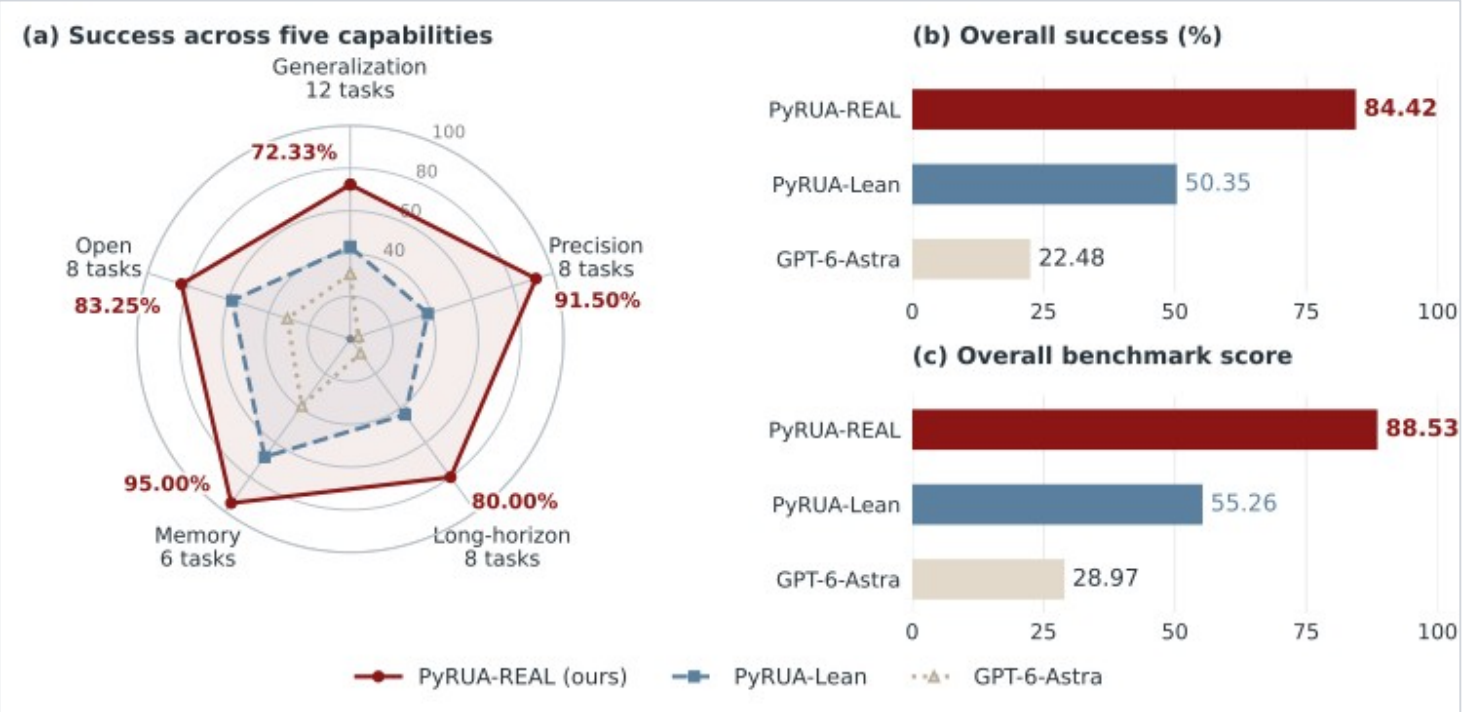
Exception: cable needs online backup

Script only 0/4 vs online GPT-6-Astra Medium assist 10/17

One success: motion 281.8 s; inter-stage checks & decisions 220.8 s (44%, incl. human review)

BENCHMARK

RoboDojo 42 tasks: PyRUA-REAL 84.42% vs PyRUA-Lean 50.35%



Method	Generalization	Fine	Long-horizon	Memory	Open	Mean
PyRUA-REAL	**72.33**	**91.50**	**80.00**	**95.55**	**88.24**	**84.42**
PyRUA-Lean	43.17	38.25	43.75	68.33	58.25	50.35
HexaShell-Preview	34.50	32.00	31.50	81.00	55.25	46.85
GPT-6-Astra	30.50	4.00	8.25	38.67	31.00	22.48

84.42% / 88.53

Mean success / benchmark score (5 categories equally weighted)

Largest margins

Fine 91.50 vs 38.25

Long-horizon 80.00 vs 43.75

(per Figure 8 and Table 5)

Caution

§ 4.3.3 text gives Fine 74.00, Long-hor. 56.00, Gen. 41.17, contradicting the figures

Frozen programs vs online zero-shot baselines; seeds differ or unstated

DISCUSSION

Discussion: method horizon, human input, online fallback

01

Method horizon limits self-improvement

Depth/calibration errors limited geometric localization: stacking moved to local visual alignment + contact feedback, Hanoi to coarse localization + wrist camera.

General pickup improved only after a human suggested SigLIP.

→ On repeated failure, explicitly compare alternative perception / control strategies.

02

Human input shapes learned skills

A human helped once when the agent kept bypassing faulty contact detection.

Unaided, the Hanoi agent invented "push a hard-to-grasp disk into a graspable pose", later reused in tic-tac-toe.

→ Claims of "autonomous improvement" must state human contributions.

03

Offline programs need online fallback

Frozen programs handle only the perception, control and recovery cases they encode.

Cable insertion success still relies on inter-stage online decisions.

→ Efficient offline execution + online agent when needed; when to invoke it remains open.

ASSESSMENT

Verdict: solid real-robot RSI evidence, but read headline numbers carefully

Strengths

+ Full "explore → improve → deploy" loop on real robots, not just final validation

+ Researcher / Engineer protocol + 3-tier experience + frozen programs: a clear, reusable paradigm

+ Rare evidence scale: tens to hundreds of hours of real development logs and diagnostics

+ Valuable counterintuitive findings: approval roles slow progress; many failures lie in verifiers; deployment time goes to waiting on thinking

+ Candid writing: notes uncontrolled comparisons, visual estimates, cross-version pooling

Limitations & concerns

– 91.1% includes online-assisted results and pools versions; no systematic repeated tests of frozen real-robot programs

– Inconsistent RoboDojo numbers (84.4 / 84.82 / 84.42; text breakdown ≠ figures; conflicting seed info)

– Unequal comparison: per-task offline-developed frozen programs vs online zero-shot baselines

– H2 sequential phases, H4 confounded by extra exploration, H3 one session per condition: all preliminary

– Costly: Push T first success ~45 h; cable insertion unsolved in 240 h; human hints used

TAKEAWAYS

Five lessons for organizing embodied RSI

1	**Rich execution feedback**	Video + joint trajectories make fixes more effective and faster (54.2% vs 15.3%)
2	**Specify task variation**	Object, layout and pose ranges steer programs toward robustness (+11 to +37 pp)
3	**Keep route re-evaluation**	Cap local fixes and escalate to the Researcher when stuck; avoid veto-only approvers
4	**Accumulate & reuse skills**	Primitive skill library: new-task success 21.3% → 40.0%, no negative transfer
5	**Freeze; fall back online if needed**	5–11× faster execution; precise contact (cable insertion) still needs an online agent

Bottom line: real-robot trial and error can become reusable, frozen robot code, but mind how "91% / 84%" are computed.